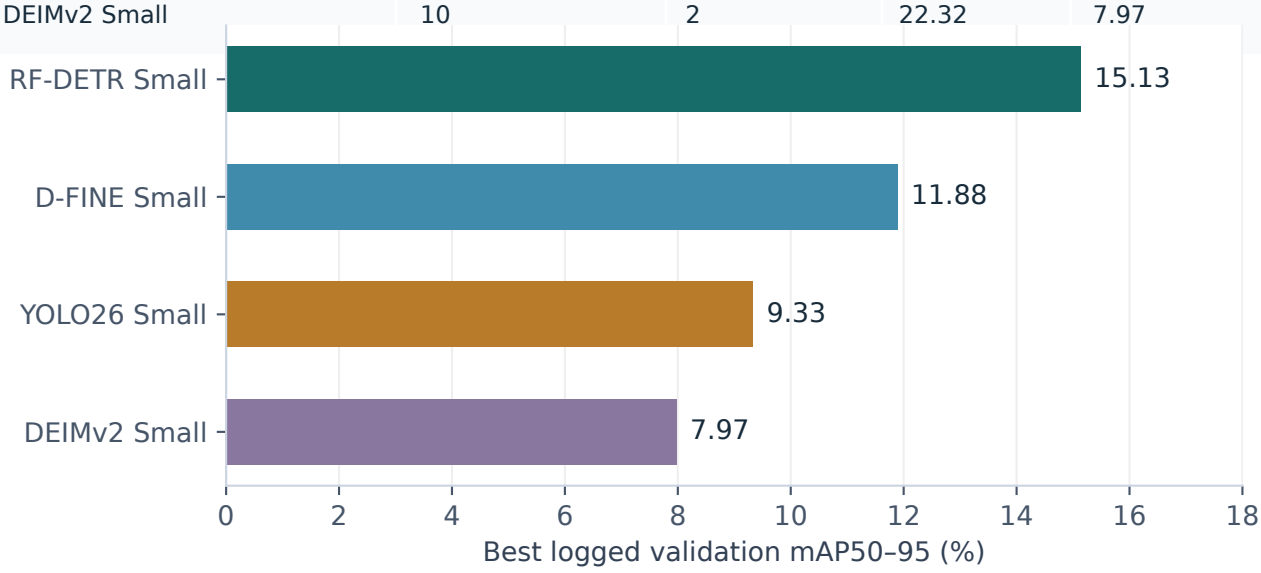
Four models, one local task

Saved experiment results • No training or inference was run for this report

Main finding

All four models have trained checkpoints and validation logs. RF-DETR Small has the highest best training-time validation mAP50–95 (15.13%), followed by D-FINE (11.88%), YOLO26 (9.33%) and DEIMv2 (7.97%). This ranks the saved experiments, not the architectures under equal training conditions.

Model	Logged epochs	Best epoch	mAP50	mAP50–95
RF-DETR Small	60	20	40.21	15.13
D-FINE Small	18	9	29.65	11.88
YOLO26 Small	14	4	27.31	9.33
DEIMv2 Small	10	2	22.32	7.97



What the records can establish

RF-DETR also has a separate, matched before/after evaluation: local validation mAP50–95 improves from 5.55% for the HRP4K base to 13.77% at epoch 20. Only RF-DETR has saved training-set mAP; the other three record training losses instead.

All four runs were configured for 100 epochs, but the selected histories contain 60, 18, 14 and 10 completed validation epochs. Logs alone do not establish why each run stopped. Epoch numbers throughout this report count from 1.

Data and measurement

What is being measured, and which settings are comparable

Dataset represented in the saved experiments

Split	Image entries	Interpretation
Local training	2,000	162 unique local images, repeated
HRP4K training	2,000	Selected HRP4K training images
Combined training	4,000	7,861 annotated pothole boxes
Local validation	41	132 pothole boxes; no training repeats
Local dataset total	203	571 annotated pothole boxes

The code prepares an 80/20 local split using seed 42. The current prepared training data repeats local examples to balance them against HRP4K. Repeated entries are not new unique observations. The model adapters reuse the same prepared data; D-FINE and DEIMv2 run manifests record matching annotation hashes.

Detection metrics, rather than a single accuracy

IoU = intersection area / union area of the predicted and target boxes. mAP50 summarizes precision–recall performance with matches requiring IoU ≥ 0.50. mAP50–95 averages AP at IoU 0.50, 0.55, …, 0.95 and therefore tests box placement more strictly. Higher is better. This task has one target class: pothole.

Precision measures the fraction of predicted potholes that are correct; recall measures the fraction of annotated potholes detected. F1 is their harmonic mean. Training loss is an optimization quantity and cannot be converted into a percentage accuracy or compared directly across architectures.

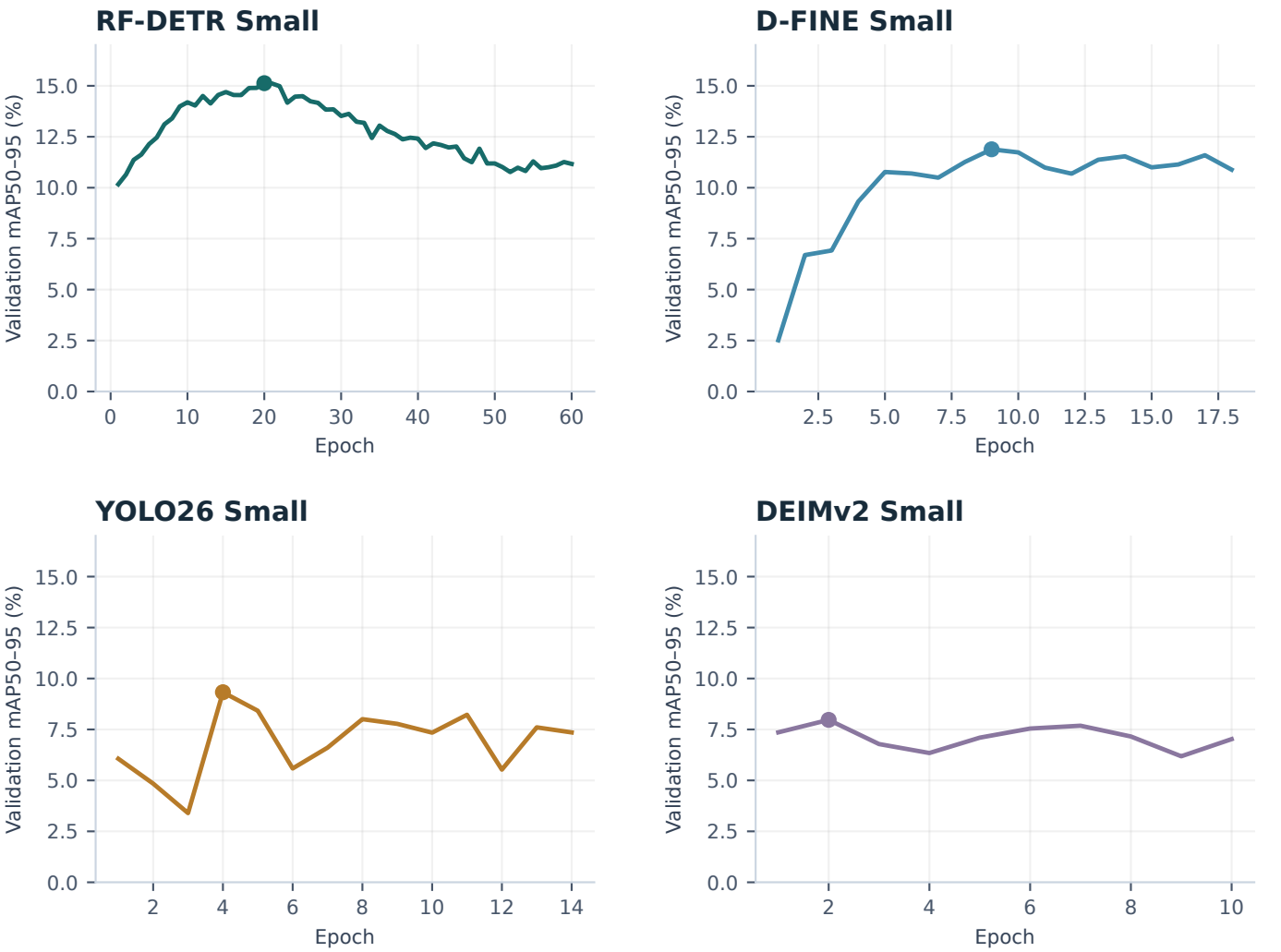
Resolved run settings

Setting	RF-DETR	D-FINE	YOLO26	DEIMv2
Input setting	896 × 896	896 × 896	896 pixels	896 × 896
Batch size	32	32	64	32
Initial learning rate	0.00001	0.0002	0.001	0.0005
Optimizer	AdamW	AdamW	AdamW	AdamW
Frozen features	Encoder	Backbone	Modules 0–10	Backbone
Configured epochs	100	100	100	100

Important: these are different training recipes and pretrained starting points. Framework validation settings also differ. For example, RF-DETR training records eval_max_dets=500, while its separate evaluator limits predictions to 100. Treat cross-model training-log scores as descriptive results.

Training histories

Validation mAP50–95 across the selected run for each model



Model	Best epoch	Best score	Last epoch	Last score
RF-DETR Small	20	15.13	60	11.17
D-FINE Small	9	11.88	18	10.89
YOLO26 Small	4	9.33	14	7.35
DEIMv2 Small	2	7.97	10	7.03

Each curve uses that framework’s own validation log. All four peak before the last logged epoch. These trajectories are consistent with a generalization or optimization limitation, but they do not by themselves prove overfitting or show how a fully tuned training run would rank.

Training versus validation

Available detection scores and the limits of the saved measurements

What is available for all four models

Model	Train mAP50 / 50–95	Val mAP50 / 50–95
RF-DETR, epoch 20	53.39 / 25.15	36.84 / 13.77
D-FINE, epoch 9	Not recorded	29.65 / 11.88
YOLO26, epoch 4	Not recorded	27.31 / 9.33
DEIMv2, epoch 2	Not recorded	22.32 / 7.97

RF-DETR’s row uses its separate evaluation for a matched train/validation comparison. The other rows use training-time validation logs. Their training mAP is missing, not zero; training losses do not fill this gap. RF-DETR’s training score here covers local repeated entries, not the combined local + HRP4K training set.

RF-DETR: matched base versus fine-tuned evaluation

Split	mAP50: base → FT	mAP50–95: base → FT	Δ 50–95 (pp)
Local train	15.78 → 53.39	4.43 → 25.15	+20.72
Local validation	15.91 → 36.84	5.55 → 13.77	+8.22
HRP4K train subset	74.68 → 75.43	45.14 → 44.49	-0.65
HRP4K val subset	52.53 → 53.31	26.57 → 26.18	-0.40

Local validation improves by 20.93 percentage points in mAP50 and 8.22 points in mAP50–95. At epoch 20, the local training–validation gap is 16.55 points in mAP50 and 11.38 points in mAP50–95. These gaps indicate stronger performance on seen images; repeated training examples affect their weighting.

Evaluation population and interpretation

Split	Entries evaluated	Pothole boxes
Local train	2,000 (162 unique images)	5,403 (includes repeats)
Local validation	41	132
HRP4K train subset	200 (10% of prepared HRP4K train)	246
HRP4K val subset	135 (15% of official validation)	152

Local validation helped select the fine-tuned checkpoint; HRP4K validation previously informed the base checkpoint. Neither is a new independent final test. HRP4K validation mAP50–95 is nearly unchanged (26.57 → 26.18), so the saved evaluation shows local adaptation without a corresponding HRP4K AP gain.

RF-DETR evaluation details

Full saved per-checkpoint detection metrics, using one evaluator

B = original HRP4K base; E20 = checkpoint_19.ckpt; E21 = checkpoint_20.ckpt. E20 is the best epoch in the training-time validation history. E21 is an additional evaluated checkpoint, not the selected primary result.

Model	Split	AP50	AP50–95	AP75	AR100	Prec.	Recall	F1
B	Local train	15.78	4.43	1.30	23.75	19.16	23.14	20.96
B	Local val	15.91	5.55	3.97	25.91	16.40	23.48	19.31
B	HRP train	74.68	45.14	47.55	59.07	48.99	78.86	60.44
B	HRP val	52.53	26.57	24.10	46.64	39.91	58.55	47.47
E20	Local train	53.39	25.15	18.59	49.54	23.93	70.59	35.74
E20	Local val	36.84	13.77	6.40	33.64	18.44	59.09	28.11
E20	HRP train	75.43	44.49	45.30	56.87	46.44	82.11	59.32
E20	HRP val	53.31	26.18	21.22	44.08	36.86	61.84	46.19
E21	Local train	55.73	25.95	18.16	49.51	24.99	71.98	37.10
E21	Local val	36.91	13.49	4.82	33.41	19.55	59.85	29.48
E21	HRP train	75.76	44.73	44.98	57.32	46.22	82.11	59.15
E21	HRP val	53.47	26.28	21.34	43.49	36.84	59.87	45.61

Detection counts at confidence ≥ 0.20, IoU ≥ 0.50

Local validation model	True positives	False positives	Missed potholes
Base	31	158	101
Epoch 20	78	345	54
Epoch 21	79	325	53

At epoch 20, detections of annotated potholes rise from 31 to 78 out of 132, while false positives rise from 158 to 345. Recall improves from 23.48% to 59.09%, but precision remains low (16.40% → 18.44%). This threshold therefore produces many false alarms despite the AP improvement.

Fixed evaluation protocol

Saved settings: CUDA; input resolution 896; maximum 100 pothole detections per image; AP uses a confidence floor of 0.0; precision/recall/F1 use confidence ≥ 0.20 and IoU ≥ 0.50 with confidence-ordered one-to-one matching. Sampling seed is 42. Confidence curves were saved from 0.00 to 1.00 in 0.01 steps.

Why the earlier RF-DETR numbers differ: the training logger reports 40.21 / 15.13 at epoch 20; this evaluator reports 36.84 / 13.77 for that epoch checkpoint. The evaluation paths and settings differ; the contribution of each difference has not been isolated. These scores should not be mixed to calculate a before/after improvement.

Base models and next comparison

What can be compared now, and what requires an additional baseline

Meaning of “base” in this project

Model	Starting weights	Base pothole class
RF-DETR Small	HRP4K pothole-trained	Supported
YOLO26 Small	COCO-pretrained yolo26s.pt	Unsupported
D-FINE Small	COCO-pretrained dfine_s_coco.pth	Unsupported
DEIMv2 Small	COCO-pretrained DINOv3-S detector	Unsupported

A COCO base has no trained pothole class. Reporting a zero pothole score would mainly express this missing class, not measure a fair transfer-learning gain. Relabeling another COCO class as pothole is invalid. A newly inserted random pothole head is an initialization control, not a trained pothole baseline.

Proposed evaluation using the saved checkpoints

1. Freeze the evaluation inputs.

Use the same 41 local validation images for all four models. Evaluate local training separately using unique source images, and optionally include the same cached HRP4K subsets. Keep exact image manifests and annotation hashes.

2. Use one scoring implementation.

Export original-image boxes, pothole labels and confidence scores from each runtime, then score with a common COCO-style evaluator. Standardize input-size setting and maximum detections; preserve model-required normalization and record resize/postprocessing differences.

3. Report a compact comparison.

For each model and split, record mAP50, mAP50–95, precision, recall, F1, TP/FP/FN and checkpoint identity. Use low confidence filtering for AP. For threshold metrics, use a stated common threshold or validation-selected thresholds applied to a separate test set.

4. Keep baseline claims explicit.

Include RF-DETR’s existing before/after comparison. Mark the other original bases as lacking a pothole class. For a meaningful before/after pothole comparison for every architecture, first obtain or train pothole-capable bases, for example on HRP4K only, and then adapt them to local data.

5. Reserve a final test set.

The current 41 images select checkpoints and provide validation estimates. A new untouched test set would support a stronger generalization claim. Do not select checkpoints or confidence thresholds using that test set.

Implementation scope: extend src/rfdert/evaluate_rfdetr_small.py with prediction adapters for the other runtimes. The existing comparison viewer is visual, not a quantitative evaluator, and rejects unsupported pothole bases. Scoring saved fine-tuned checkpoints needs inference, not retraining. Runtime speed and memory were not benchmarked in these records.

Evidence and reproducibility

Local files behind the tables and charts

This report summarizes local artifacts inspected on 09 November 2026. It does not add measurements, download data, install dependencies, or change training code. Source files under out/ are ignored by Git, so preserving the report alone does not preserve the original checkpoints and logs.

Selected runs and checkpoint files

Model	Run timestamp	Metrics file	Checkpoint
RF-DETR	2026-11-02_19-28-35_676993	metrics.csv	checkpoint_19.ckpt
D-FINE	2026-11-04_20-01-02_456706	metrics.csv	best_stg1.pth
YOLO26	2026-11-03_13-25-47_299595	results.csv	weights/best.pt
DEIMv2	2026-11-04_22-01-20_947361	log.txt	best_stg1.pth

Run directories: out/hrp4k-rfdetr-small-finetuned/ (RF-DETR), out/dfine-small-finetuned/, out/yolo26-small-finetuned/ and out/deimv2-small-finetuned/. Append the run timestamp above to locate its metrics and checkpoints.

Matched RF-DETR evaluation

Within the RF-DETR run: evaluation/2026-11-02_21-17-39_643070/

results.json records settings and all 12 model/split rows; metrics.csv mirrors the scores. splits.json stores the evaluated images and boxes. curves.json / curves.csv store threshold sweeps. training_config.json records RF-DETR's resolved training settings.

Other supporting evidence

YOLO: args.yaml. D-FINE / DEIMv2: run.json, resolved_config.yaml and log.txt. Dataset counts: data/annotations/original_coco.json and data/finetune_hrp4k/{train,valid}/_annotations.coco.json. Baseline handling: src/detr_worker.py, src/yolo/config.py and src/rfdert/evaluate_rfdetr_small.py.

Source fingerprints (SHA-256, first 16 hex characters)

Source	Fingerprint
RF-DETR training CSV	4fd1e17bf24445d4
YOLO training CSV	a653cb5ca9d4f260
D-FINE training CSV	b3e304de6a26cfb1
DEIMv2 training log	17343c6f5639a65b
RF-DETR evaluation JSON	d13429d34ff52892

Interpretation limits: no independent test set, no saved train mAP for three models, different training durations/recipes, different base label spaces, and only framework-specific validation for the cross-model ranking. Uncertainty intervals and inference latency are not available. Missing scores are explicitly labeled rather than inferred.